

Учебник по скриптингу в Affinity (SDK 33000)

JavaScript-скрипты для Affinity Designer / Photo / Publisher. Источник:

docs/textbook/chapters/. Онлайн-SDK: <https://sdk.affinity.studio/33000/js/index.html>

Референс API: docs/sdk/. Гайды: docs/guides/.

Глава 1. Как выполняются скрипты

1.1. Что такое скрипт Affinity

Скрипт — синхронный JavaScript, который выполняется **внутри** Affinity (Designer / Photo / Publisher). Вы отправляете код через MCP-инструмент `affinity_execute_script`, Affinity выполняет его и возвращает текст из `console.log()`.

Три следствия:

1. **Нет return.** Результат — только `console.log()`.
2. **Нет module.exports.** Файл должен выполняться сразу, сверху вниз.
3. **Первая строка — "use strict".**

1.2. Самый маленький скрипт

```
"use strict";  
console.log('Hello from Affinity!');
```

Выполните его через `affinity_execute_script` — в ответ придёт строка `Hello from Affinity!`.

1.3. MCP-сервер

Сервер работает на `http://localhost:6767` (конфиг `.mimocode/config.json`). Инструменты: выполнение, чтение документации SDK, библиотека скриптов, рендер спреда/выделения, поиск подсказок. Полный список — в `guides/mcp-tools.md`.

Обязательное начало работы:

```
affinity_read_sdk_documentation_topic(filename="preamble")
affinity_search_sdk_hints(prompt="ваша задача")
```

1.4. Ограничения среды

- Файловая система — **только рабочий стол** (`app.userDesktopPath`).
- Рендер результата — JPEG не больше 1024px.
- Если команда вернула NOT_ALLOWED — пользователь запретил AI/файлы/сеть в настройках Affinity. Это не баг скрипта.
- Скрипты синхронные: *Async-варианты методов SDK вам не нужны.

Упражнения

1. Выполните hello-скрипт и найдите его вывод в ответе инструмента.
2. Прочитайте preamble через `affinity_read_sdk_documentation_topic`.
3. Выполните `affinity_list_sdk_documentation` и найдите темы `document.js` и `dialog.js`.

Глава 2. Документ и среды

2.1. Текущий документ

```
"use strict";
const { Document } = require('/document.js');
const doc = Document.current;           // активный документ (DocumentApi.getCurrent
if (!doc) { console.log('No document open'); return; }
console.log('Title: ' + doc.title);      // свойство JSlib над getTitle()
console.log('Spreads: ' + doc.spreads.length);
console.log('Units: ' + doc.units + ' DPI: ' + doc.dpi);
```

`Document.current` нет — скрипт вежливо завершается через `console.log`, а не падает с исключением. Так начинается **каждый** скрипт.

Другие способы получить документ: `Document.enumerateOpen()` (все открытые), `Document.load(path)` (открыть с Desktop). Создание: `createFromOptions`, `createFromPreset` (`affinity:dom`, см. `sdk/dom-document.md`).

2.2. Текущий спред

```
"use strict";
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const doc = Document.current;
if (!doc) { console.log('No document open'); return; }
const spread = doc.currentSpread;      // свойство над getCurrentSpread()
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));
```

Вызов `createSetCurrentSpread` **обязателен перед любыми правками**: без него цель вставки игнорируется и новые узлы ложатся плоско на документ (реальный баг `chart-builder 1.5.0`, исправлен в 1.5.1).

2.3. Все спреды документа

```
for (const spread of doc.spreads) {
  console.log('Children: ' + spread.children.length);
}
```

Пакетные скрипты (типа `hanging-chars`) обходят **все** спреды, а не только текущий.

Упражнения

1. Выведите название, путь (`doc.path`) и флаг изменений (`doc.isDirty`) открытого документа.
2. Посчитайте суммарное число дочерних узлов по всем спредам.
3. Откройте второй документ и перечислите оба через `enumerateOpen()`.

Глава 3. Узлы и дерево документа

3.1. Что такое узел

Всё на спреде — узлы: фигуры, текстовые фреймы, изображения, группы, направляющие. У каждого узла есть дети (`node.children`) — получается дерево. Тип узла проверяется флагами: `node.isTextNode`, а имя для человека — `node.userDescription` (именно по нему `chart-builder` ищет группы `line chart`).

3.2. Обход дерева

Два способа — рекурсия и явный стек:

```
"use strict";
// Рекурсия (коротко):
function walk(node, fn) {
  fn(node);
  if (node.children) for (const ch of node.children) walk(ch, fn);
}
for (const child of spread.children) walk(child, (n) => { /* ... */ });

// Явный стек (hanging-chars обходит ВСЕ спреды именно так):
for (const spread of doc.spreads) {
  const stack = [...spread.children];
  while (stack.length) {
    const node = stack.pop();
    if (node.isTextNode) { /* ... */ }
    if (node.children) for (const ch of node.children) stack.push(ch);
  }
}
```

3.3. Родители и корень

Узел знает родителя (`node.parent`), документ — корневой узел (`doc.rootNode`). Полный перечень типов узлов и их методов — `sdk/dom-nodes.md`, контентных (фигуры, текст, картинки) — `sdk/dom-content.md`.

Упражнения

1. Соберите статистику: сколько узлов каждого типа на спреде (различайте хотя бы текстовые: `isTextNode` — и все остальные).
2. Найдите самый глубоко вложенный узел и выведите глубину.
3. Найдите все группы с `userDescription === 'line chart'`.

Глава 4. Команды, undo и preview

4.1. Золотое правило: мутации только командами

Узлы **нельзя** менять напрямую присваиванием. Любое изменение — это команда, выполненная через документ:

```
doc.executeCommand(какаяТоКоманда);
```

Команды создаются фабриками `DocumentCommand.create*` — их 363 штуки, полный каталог: `sdk/commands-catalog.md`. Узлы для команды заворачиваются в выделение: `Selection.create(doc, nodes)` (`sdk/dom-selections.md`).

4.2. Один undo-шаг на батч

Создание нескольких узлов упаковывается в `AddChildNodesCommandBuilder` (`sdk/commands.md`): один `executeCommand` — один шаг `Ctrl+Z`. Разнородные изменения — в `CompoundCommandBuilder` (пример: `hanging-chars` кладёт десятки `createSetText` в один undo-шаг на текстовое поле).

```
"use strict";
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const compound = CompoundCommandBuilder.create();
compound.addCommand(DocumentCommand.createSetText(sel, ' '));
doc.executeCommand(compound.build());
```

4.3. Preview без порчи документа

Интерактивные скрипты показывают результат до нажатия ОК:

```
const historyStart = doc.history.position;
doc.executeCommand(cmd, true); // true = предпросмотр
// OK:      createClearPreviews() + финальное применение
// Cancel:  createClearPreviews() + doc.history.position = historyStart
```

4.4. Чтение результата команды

После `builder.createCommand(...)` созданные узлы доступны как `[...cmd.newNodes][0]` — так `chart-builder` и `color-palette-gen` забирают группу, чтобы потом сложить в неё содержимое вторым

батчем.

Упражнения

1. Объясните, почему прямой `node.userDescription = 'x'` не работает.
2. Соберите compound из двух команд и убедитесь, что `Ctrl+Z` откатывает обе сразу.
3. Реализуйте preview: примените команду с `true`, затем откатите через history.

Глава 5. Фигуры и геометрия

5.1. Фигуры как данные

Фигура (`affinity:geometry`, `sdk/geometry.md`) — это параметры: `ShapeRectangle`, `ShapeEllipse`, `ShapeStar`, `ShapeArrow`, `ShapePie`, `ShapeQRCode` и ещё ~25 типов (`ShapeType`). Сырые фабрики: `Shape.create(type)`, у каждого типа — свои `create()/get*/set*` (например, `ShapeArrowApi`: `setThickness`, `setLeftLength`, ...).

В скриптах фигуры обычно создаются определениями узлов (`ShapeNodeDefinition` из `/nodes.js`) и кладутся на спред батчем (глава 4). Параметры углов, заливки и обводки задаются там же.

5.2. Кривые

`CurveBuilder` строит `PolyCurve` — произвольные контуры, стрелки серий на графиках, точки данных. Важный порядок аргументов:

```
// PolyCurveNodeDefinition.create(polyCurve, brushFill, lineFill, lineStyle, ...)
// lineStyle и lineFill путать НЕЛЬЗЯ: ошибка 'expected FillDescriptorHandle'
// роняет скрипт молча (реальный случай chart-builder 1.4.1).
```

Серийные линии графиков — простые, без наконечников.

5.3. Трансформации

`Rectangle + Transform`: положение, размер, поворот (`sdk/geometry.md`: `RectangleApi`, `TransformApi`). Палитра `color-palette-gen` вычисляет место справа от artwork: `x = ext.x + ext.width + 50`.

Упражнения

1. Перечислите все значения ShapeType (подсказка: sdk/geometry.md + енам).
2. Создайте ShapeArrow и поменяйте thickness через сеттер.
3. Постройте PolyCurve из трёх точек и добавьте на спред батчем.

Глава 6. Заливки, цвета, градиенты

6.1. Дескрипторы заливки

```
"use strict";  
const { FillDescriptor, SolidFill, FillType, GradientFill, GradientFillType } = require('...');  
const { Colour, ColourProfileSet } = require('/colours.js');
```

У узла две заливки: кисть (hasBrushFill → brushFillDescriptor) и перо (hasPenFill → penFillDescriptor, плюс lineWeightPts — толщина обводки). Градиент с новым трансформом — клоном: cloneWithNewTransform (так color-palette-gen рисует свотчи 120×40).

6.2. Сканирование цветов спреда

Паттерн из color-palette-gen (протестирован): рекурсивный обход, три словаря fills / strokes / gradients, ключ r,g,b для сплошных и fill|<type>|stops для градиентов, прозрачность — флаг α:transp. Нечитаемые узлы пропускаются через try/catch.

6.3. Метки значений

Свотч 40×40 + подпись из четырёх строк: RGB, CMYK (через ColourProfileSet.default, при ошибке C: ? M: ? Y: ? K: ?), HSL, HEX. У обводок — пятая строка Width: Xpt. У градиентов — тип, число стопов и position → HEX по каждому стопу.

Упражнения

1. Соберите все уникальные сплошные заливки текущего спреда в console.log.
2. Найдите все обводки толще 2pt.
3. Перечислите стопы первого найденного градиента.

Глава 7. Текст: Story, глифы, абзацы

7.1. Анатомия текста

Текстовый узел (isTextNode) — это рамка. Внутри: storyInterface.story — Story, у неё getText(begin, length) для чтения. Форматирование бывает двух уровней: GlyphAtts (символ: шрифт, размер, цвет) и ParagraphAtts (абзац: выравнивание, интервалы). Полный референс: sdk/story-text.md (34 API, 332 метода).

7.2. Чтение

```
const originalText = story.getText(range.begin, range.end - range.begin);
```

Так hanging-chars читает каждое текстовое поле на каждом спреде.

7.3. Точечная запись

Запись — только командой createSetText через TextSelection:

```
"use strict";
const { Selection, TextSelection } = require('/selections.js');
const sel = Selection.create(doc, node);
sel.addSubSelectionForNode(/* TextSelection: начало range.begin + pos, длина 1 */);
compound.addCommand(DocumentCommand.createSetText(sel, ' '));
```

NBSP (U+00A0) вместо обычного пробела после одиночных букв и предлогов — разбор трёх паттернов (findOrphanSpacePositions) см. в hanging-chars.

7.4. Построение текста с нуля

StoryBuilder.addText(utf8Text) (сигнатура сверена с онлайн-SDK), разметка — GlyphAtts. Так color-palette-gen печатает подписи свотчей, а chart-builder — подписи осей и легенды.

Упражнения

1. Выведите первые 200 символов каждого текстового поля документа.
2. Найдите все позиции ...на ... и замените пробел на NBSP одним compound.
3. Создайте текстовый фрейм через `FrameTextNodeDefinition` + `StoryBuilder`.

Глава 8. Выделения

8.1. Зачем Selection

Команды не принимают «голые» узлы: их заворачивают — `Selection.create(doc, nodes)` (`sdk/dom-selections.md`, 14 API). Подвыделения уточняют цель: `TextSelection` (диапазон текста), кривые (`CurveNodeSubSelection`, `CurveEdgeSubSelection`), таблицы (`TableSubSelection`), заливки и прозрачности.

8.2. Типичные связки

Задача	Связка
Заменить символ	<code>Selection + TextSelection + createSetText</code>
Удалить узлы	<code>Selection + createDeleteNodesCommand</code>
Переместить в группу	<code>Selection + createMoveNodes (NodeMoveType.Inside)</code>
Текущее выделение пользователя	<code>doc.selection / getCurrentSelection</code>

8.3. Проверка перед действием

`selection.isEmpty()`, `itemCount`, `firstNode` — Bail out, если пользователь ничего не выделил (`chart-builder` требует выбранный `FrameText`-узел и показывает `Select a text frame`, если его нет).

Упражнения

1. Выведите число узлов в текущем выделении пользователя.
2. Программно выделите все текстовые узлы спреда.
3. Переместите выделенное в новую группу одним undo-шагом.

Глава 9. Диалоги и UI

9.1. Показ диалога

```
"use strict";
const { Dialog } = require('/dialog.js');
// const result = dialog.runModal();
// if (result === DialogResult.OK) { ... }
```

`Dialog.show()` deprecated — только `runModal()` (`sdk/ui-dialogs.md`, 24 API). Контролы: `ComboBox`, `CheckBox`, `Switch`, `TextBox`, `UnitValueEditor`, `ColourPicker`, `FillEditor/StrokeEditor`, `FontPicker`, `RadioGroup`, `Button/ButtonSet`, `StaticText` (`DialogResult.OK` — подтверждение).

9.2. Диалог `chart-builder` как образец

Заголовок `Chart Builder`, ширина 350: тип (`Line/Bar/Donut`), размеры W/H (дефолт 500×400, $\leq 500/400$), толщина, сетка (2–20), радиус углов (`Bar`), по `ColourPicker` на серию из палитры 32 цветов, режим подписей, легенда, проценты для `Donut`. Отмена → `Cancelled` в консоль, без созданных узлов.

9.3. Диалог-отчёт без настроек

`hanging-chars` показывает один модальный итог: `Fixed N of M text field(s)...`, `Everything already looks correct.`, `No text fields found.` + `Errors: ....` Ошибки отдельных узлов не роняют весь проход — копят в отчёт.

Упражнения

1. Соберите диалог с `TextBox` и кнопками `OK/Cancel`, верните ввод в `console.log`.
2. Добавьте `ColourPicker` и покрасьте им выделенные фигуры.
3. Реализуйте связку `Preview/Cancel` из главы 4 через диалог.

Глава 10. Растр, эффекты, AI

10.1. Растровые узлы

Модуль `affinity:raster` (`sdk/raster.md`, 22 API): пиксельный доступ (`PixelReader*`/`PixelReaderWriter*` — по форматам RGBA8/CMYKA8/LABA16/...), выделения (`RasterSelectionApi`), объекты (`RasterObjectApi`). Сами растровые узлы документа (фильтры, коррекции) живут в `affinity:dom` (`sdk/dom-nodes-raster.md`, 156 API): размытия, шумы, уровни, HSL — у каждого тройка `*Parameters` / `*RasterNode` / `*RasterNodeDefinition`.

10.2. Эффекты слоёв

`affinity:layereffects` (`sdk/linestyles-effects.md`, 12 API): тени (внутренняя/внешняя), свечения, барельеф, наложения цвета и градиента, гауссово размытие. Дублирование эффекта на другой слой — команды `createDuplicate*LayerEffectCommand` из каталога команд.

10.3. AI-команды

Генерация и обработка изображений (`generateImage`, `generativeEditImage`, `removeBackground`, `selectSubject`, `detectDepth`, `colourise`, `imageTrace`). Помните про `NOT_ALLOWED`: пользователь может запретить AI в настройках — обрабатывайте такой ответ, а не считайте его багом.

Упражнения

1. Перечислите все `*LayerEffect`-команды дублирования в каталоге.
2. Прочитайте пиксели выделенного растра через подходящий `PixelReader`.
3. Примените `removeBackground` и проверьте ответ на `NOT_ALLOWED`.

Глава 11. Файлы, сеть, таймеры, экспорт

11.1. Файлы: только Desktop

```
// const { ... } = require('/fs.js'); // FileApi, FileSystemApi (sdk/fs-network-c
const { app } = require('/application.js');
console.log(app.userDesktopPath); // единственный доступный корень
```

Любой путь вне Desktop — ошибка. Бинарные данные — через `/buffer.js`.

11.2. Сеть

HttpRequest/HttpResponse (affinity:network, методы RequestMethod, статусы HttpStatusCode): tableFromJson тянет JSON для таблицы именно так. Сеть тоже попадает под NOT_ALLOWED.

11.3. Экспорт

Документ: doc.export(...) / exportAsync, конфиг — ExportConfigApi, форматы/масштабы — ExportFormatApi/ExportScaleApi/ExportSizeApi (sdk/dom-document.md). Шаблон: templates/export.js. Проверка результата — affinity_render_spread.

11.4. Таймеры

affinity:timers (TimerApi, 11 методов) — отложенные и повторные задачи. В синхронных скриптах применяются точно.

Упражнения

1. Сохраните лог работы скрипта файлом на Desktop (пример: logToFile.js).
2. Загрузите JSON по HTTP и постройте из него таблицу (пример: tableFromJson.js).
3. Экпортируйте текущий спред и проверьте файл.

Глава 12. Отладка и разбор трёх боевых скриптов

12.1. Методика отладки

1. console.log после каждого этапа (счётчики: Fills: N Strokes: M).
2. affinity_render_spread — глазами быстрее, чем гадать.
3. Минимальный репро: режьте скрипт пополам, пока не найдёте падающую строку.
4. affinity-check: use strict, module.exports, пути без .js, preview без createClearPreviews.
5. Найденное решение — сразу в affinity_add_sdk_hint.

Типовые падения: нет setCurrentSpread; перепутан порядок (polyCurve, brushFill, lineFill, lineStyle, ...); require без .js; мутация узла напрямую; диалог через show().

12.2. hanging-chars: проход по всем текстам

Задача: NBSP после одиночных букв и предлогов (RU/EN, строгие списки 2–3 букв). Схема: все спреды → стек → `isTextNode` → `getText` → `findOrphanSpacePositions` (3 паттерна, дедупликация, сортировка) → один `compound createSetText` на поле → диалог-отчёт. Undo: шаг на поле. Файл: `scripts/hanging-chars/source/hanging-chars.js`, спека: `docs/specs/hanging-chars.md`.

12.3. color-palette-gen: палитра использования

Задача: свотчи всех fills/strokes/gradients справа от artwork. Схема: обход `hasBrushFill/hasPenFill` → словари → `origin x + width + 50` → по `AddChildNodesCommandBuilder` на элемент → группировка в FILLS/STROKES/GRADIENTS через `createMoveNodes` (`NodeMoveType.Inside`). Файл: `scripts/color-palette-gen/...`, спека: `docs/specs/color-palette-gen.md`.

12.4. chart-builder: диаграммы из текста

Задача: Line/Bar/Donut из текстового фрейма (`Series: v1, v2... --- labels`). Схема: проверка `FrameText` → `parseData` → диалог (тип, W/H, сетка, цвета серий) → группа `line/bar/donut chart` (`uniqueGroupName`, коллизии → `base_2`) → геометрия батчем → `applyLineStyles` → легенда. Undo: два шага. Файл: `scripts/chart-builder/source/chart-builder.js`, спека: `docs/specs/chart-builder.md`.

Выпускная работа

Напишите свой скрипт по полному циклу: спека (`docs/specs/_template.md`) → `slice` → `affinity-check` → тесты на пустом и боевом документе → `undo-проверка` → `release/` + README. Удачи!